Section 3.3: Forward Pass and Backpropagation

From Prediction to Learning

AI-IM Class — Lecture Notes

In Section 3.2 we learned that gradient descent improves a model by repeating two things: compute the loss, then nudge every parameter against its gradient. For a neural network this cycle has two halves:

- the **forward pass**, which turns an input $\mathbf{x}$ into a prediction $\hat{y}$ and a loss J, and
- **backpropagation**, which works out how much each weight contributed to that loss, giving the gradient $\partial J/\partial w$.

We will compute the forward pass by hand, see the *idea* behind backpropagation, and then check that one update step really lowers the loss.

1 The Network

We work with a **two-layer feedforward network** (one hidden layer) for the XOR problem:

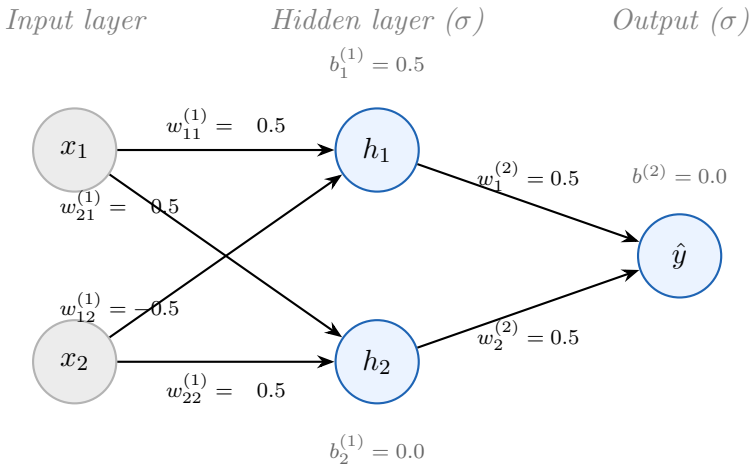


Figure 1: A two-layer feedforward network with one hidden layer.

In matrix form, the (not yet trained) parameters are

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 \\ 0.0 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix}, \quad b^{(2)} = 0.0,$$

where row i of $\mathbf{W}^{(1)}$ holds the weights *feeding into* hidden unit i.

1.1 The Sigmoid Activation

In Section 3.1 we used the **step function**. Its derivative is 0 almost everywhere, so gradient descent would get no signal about which way to move the weights. We therefore

use its smooth "S-shaped" version, the **sigmoid**:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma(z) \in (0,1).$$

Every hidden and output neuron does two sub-steps:

1. **Pre-activation** z: a weighted sum of inputs plus bias.
2. **Activation** $h = \sigma(z)$: squeeze z into $(0,1)$.

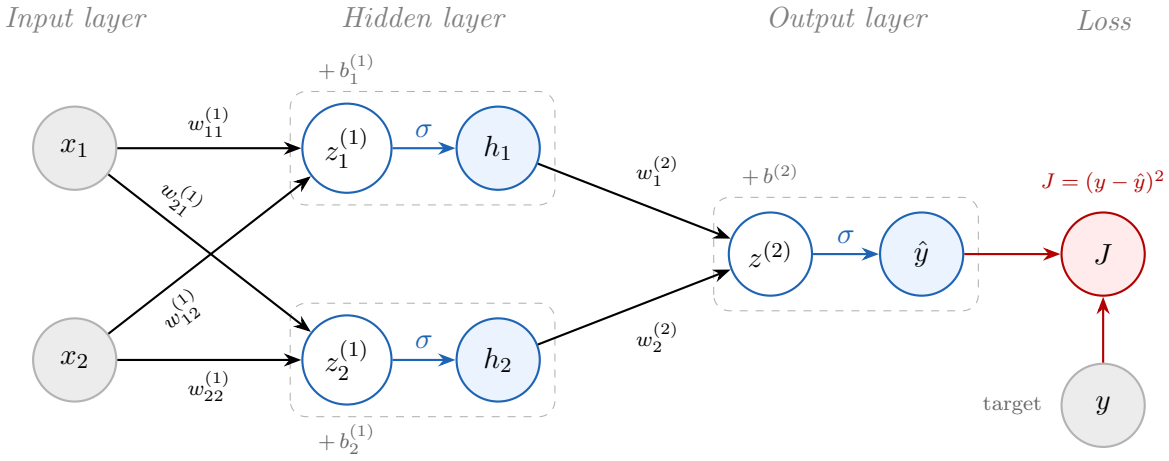


Figure 2: The same network with each neuron (dashed box) split into its pre-activation z (white) and activation (blue). The prediction $\hat{y}$ is compared with the target y to give the loss J.

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

2 The Forward Pass

We feed the XOR input $\mathbf{x} = (x_1, x_2) = (0, 1)$, whose target is $y = 1$.

Step 1 Hidden layer

Weighted sums entering each hidden neuron:

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} = (0.5)(0) + (-0.5)(1) + 0.5 = 0.0$$
$$z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} = (0.5)(0) + (0.5)(1) + 0.0 = 0.5$$

Apply the sigmoid:

$$h_1 = \sigma(z_1^{(1)}) = 0.500, \qquad h_2 = \sigma(z_2^{(1)}) = 0.622$$

Step 2 Output layer

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)} = (0.5)(0.500) + (0.5)(0.622) + 0.0 = 0.561$$

$$\hat{y} = \sigma(z^{(2)}) = 0.637$$

Step 3 Loss

Using the squared error:

$$J = (y - \hat{y})^2 = (1 - 0.637)^2 \approx 0.132$$

The network outputs 0.637 when it should output 1. A perfect prediction would give $J = 0$. Our goal now is to change the weights so that J goes down.

3 Backpropagation: The Idea

Gradient descent (Section 3.2) updates every parameter with

$$w \longleftarrow w - \eta \frac{\partial J}{\partial w}.$$

So we need $\partial J/\partial w$ for *every* weight and bias: nine numbers for our small network, and millions for a real one. The difficulty is that the loss depends on a weight only *indirectly*, through a chain of intermediate values:

$$w_{12}^{(1)} \;\to\; z_1^{(1)} \;\to\; h_1 \;\to\; z^{(2)} \;\to\; \hat{y} \;\to\; J.$$

3.1 The Chain Rule

If y depends on u, and u depends on x, then

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

Longer chains simply multiply more factors.

Example 3.1. Let $y = (2x + 1)^2$. Write $u = 2x + 1$ so that $y = u^2$:

$$\frac{dy}{dx} = \underbrace{\frac{dy}{du}}_{2u} \cdot \underbrace{\frac{du}{dx}}_{2} = 4(2x + 1).$$

3.2 Walking Backwards Through the Network

Each arrow in Figure 2 is a simple function whose derivative is easy (a weighted sum or a sigmoid). The chain rule says:

the gradient of J with respect to a weight is the product of the local derivatives along the path from J back to that weight.

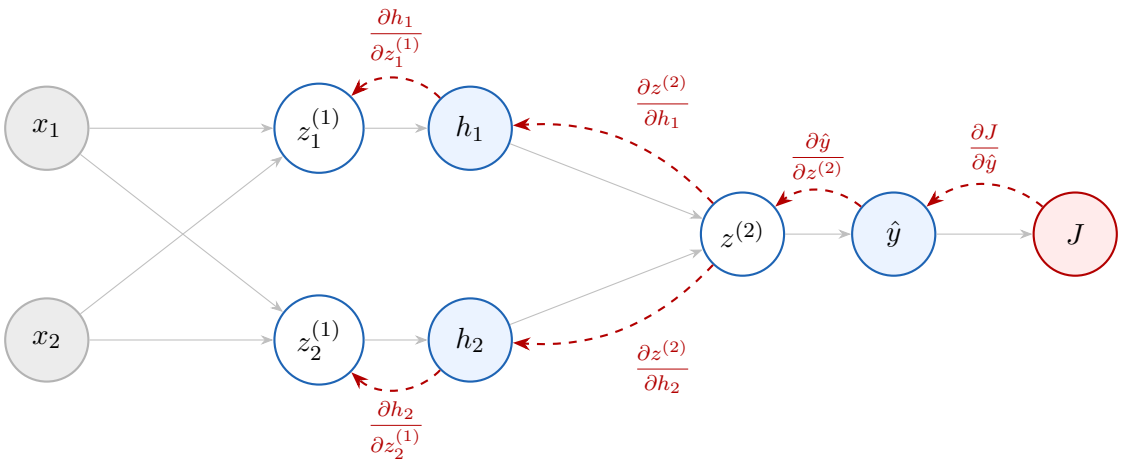


Figure 3: Backpropagation runs the forward graph in reverse. Starting from J, it multiplies the local derivative on each dashed arrow. Paths share their first links ($J \to \hat{y} \to z^{(2)}$), so those products are computed once and reused for every weight further back.

For example, the gradient for $w_{12}^{(1)}$ is the product along its path:

$$\frac{\partial J}{\partial w_{12}^{(1)}} = \underbrace{\frac{\partial J}{\partial \hat{y}}}_{2(\hat{y}-y)} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z^{(2)}}}_{\sigma'(z^{(2)})} \cdot \underbrace{\frac{\partial z^{(2)}}{\partial h_1}}_{w_1^{(2)}} \cdot \underbrace{\frac{\partial h_1}{\partial z_1^{(1)}}}_{\sigma'(z_1^{(1)})} \cdot \underbrace{\frac{\partial z_1^{(1)}}{\partial w_{12}^{(1)}}}_{x_2}.$$

What to remember about backpropagation.

1. It starts at the loss with the **error** $\dfrac{\partial J}{\partial \hat{y}} = 2(\hat{y} - y)$, i.e. "prediction minus target".
2. Moving back one layer multiplies in that layer's weight and one activation slope $\sigma'(z)$. For the sigmoid, $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) \le 0.25$.
3. At the end, each weight's gradient is

$$\frac{\partial J}{\partial w} = (\text{error signal at its neuron}) \times (\text{input to that weight}).$$

A weight whose input is 0 gets zero gradient: it had no effect on this prediction.

Nobody does this by hand. Deep-learning libraries record the forward pass and apply the chain rule automatically (*automatic differentiation*). In PyTorch, for example, a single call to `loss.backward()` fills in $\partial J/\partial w$ for every parameter.

4 One Update Step

Running backpropagation on our input $(0, 1)$ and applying $w \leftarrow w - \eta\, \partial J/\partial w$ with learning rate $\eta = 0.1$ gives the following new parameters:

Parameter	Old	Gradient $\partial J/\partial(\cdot)$	New
$w_1^{(2)}$	0.5	-0.084	0.508
$w_2^{(2)}$	0.5	-0.105	0.510
$b^{(2)}$	0.0	-0.168	0.017
$w_{11}^{(1)}$	0.5	0.000	0.500
$w_{12}^{(1)}$	-0.5	-0.021	-0.498
$b_1^{(1)}$	0.5	-0.021	0.502
$w_{21}^{(1)}$	0.5	0.000	0.500
$w_{22}^{(1)}$	0.5	-0.020	0.502
$b_2^{(1)}$	0.0	-0.020	0.002

Notice that:

- All non-zero gradients are **negative**, so every parameter that moves goes *up*, which pushes $\hat{y}$ up towards $y = 1$.
- $w_{11}^{(1)}$ and $w_{21}^{(1)}$ have zero gradient because their input is $x_1 = 0$, so they stay put.
- The output-layer gradients are larger than the hidden-layer ones: each step back multiplies in another factor $\sigma'(z) \leq 0.25$.

4.1 Did the Loss Go Down?

Run the forward pass again on the same input $(x_1, x_2) = (0, 1)$, this time with the **new** parameters.

Step 1 Hidden layer

$$z_1^{(1)} = (0.500)(0) + (-0.498)(1) + 0.502 = 0.004$$
$$z_2^{(1)} = (0.500)(0) + (0.502)(1) + 0.002 = 0.504$$

$$h_1 = \sigma(z_1^{(1)}) = 0.501, \qquad h_2 = \sigma(z_2^{(1)}) = 0.623$$

Step 2 Output layer

$$z^{(2)} = (0.508)(0.501) + (0.510)(0.623) + 0.017 = 0.589$$
$$\hat{y} = \sigma(z^{(2)}) = 0.643$$

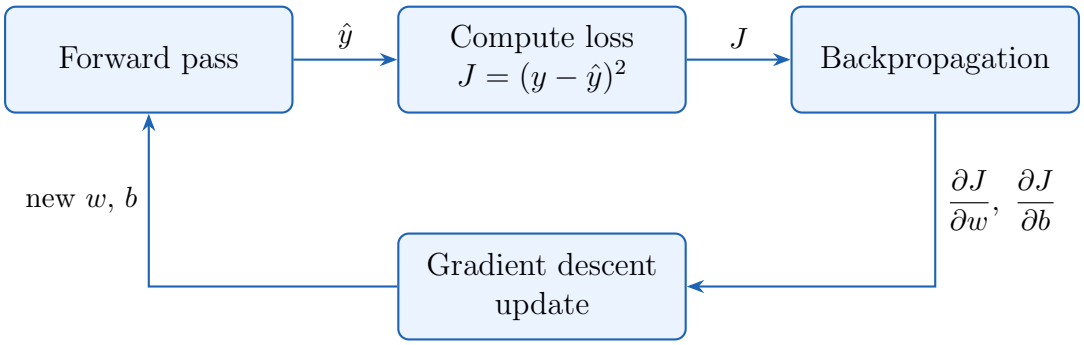
Step 3 Loss

$$J = (1 - 0.643)^2 \approx 0.127$$

Quantity	Before update	After update
Prediction $\hat{y}$	0.637	0.643
Loss J	0.132	0.127

After one step the prediction has moved closer to the target and the loss has dropped. The change is small, but repeating it many times is what training *is*.

5 The Training Loop



One forward–backward pass uses one input. In practice we cycle through all four XOR inputs (or use mini-batches), average the gradients, and take one update step per batch. After hundreds or thousands of such cycles the network correctly classifies all four XOR inputs.